

Il turno e il commit

Come lavora Lele: vigile del fuoco a turni, non programmatore di mestiere, con **quattro applicativi in produzione** costruiti in dieci mesi con un'AI come unico collaboratore. Questo documento descrive il metodo, i punti su cui si appoggia, dove regge e dove si rompe.

REPOSITORY ATTIVI 6 di cui 4 con utenti veri	COMMIT 2026 1.145 marzo → settembre	RIGHE IN PRODUZIONE 86k 34k PHP + 52k TypeScript	MIGRAZIONI DB 134 un solo gestionale, in 5 settimane
TEMPO DISPONIBILE turni orari irregolari, sessioni corte			

IL PUNTO DI PARTENZA

Da «un comando alla volta» a brief per agenti paralleli

A marzo 2026 il profilo utente scritto dall'AI diceva: *«non è un programmatore, spiegare tutto in modo semplice, un comando alla volta, aspettare conferma prima del passo successivo»*. A settembre le istruzioni permanenti dicono l'opposto: *«su codice e automazioni vai alla soluzione, Lele sa quello che fa»*, e le sessioni si chiudono con tre agenti lanciati in parallelo su aree di file diverse.

La distanza fra quelle due righe è il fatto più importante di questo documento. Non è cresciuta la capacità di scrivere codice: è cresciuta la capacità di **dirigere**, collaudare e non fidarsi. Il metodo che segue è quasi tutto difensivo, ed è nato una cicatrice alla volta.

IL CICLO DI LAVORO

Sette passaggi, sempre nello stesso ordine

Non è una convenzione teorica: è la sequenza ricostruibile dai commit e dalle schede di progetto degli ultimi tre mesi. Ogni passaggio esiste perché la sua assenza è

costata qualcosa.

01 **La segnalazione arriva dall'utente vero, dentro l'applicativo**

Un tasto flottante «Qui non va» su ogni pagina apre una nota numerata con il link alla pagina d'origine. Il furiere del comando VVF scrive lì, non su WhatsApp. Le note diventano la coda di lavoro: #257, #259, #253, #256, #258.

02 **Ricognizione prima di delegare**

Parola d'ordine dal 06/08: «massima resa per la minima spesa». Chi dirige guarda i file e prepara gli elenchi *prima* di aprire un agente, così l'agente non spende metà del suo contesto a riscoprire il progetto.

03 **Le note si raggruppano per area di file, non per priorità**

Agenda / ODT / Amministrazione: un agente per area, sfalsati nel tempo. Due agenti non toccano mai lo stesso file. Nel brief entrano i precedenti che contano — il commit da imitare, la regola già costata un incidente, il lavoro già fatto da non rifare.

04 **Collaudo con dati finti nel database vero**

Niente test automatici: uno scenario finto scritto nel DB di produzione, verificato, poi rimosso. Sulle migrazioni, dry-run con rollback. È il compromesso di chi non ha un ambiente di staging — e il punto debole numero uno.

05 **Deploy a fine sessione, sempre**

Regola fissa: il commit non basta, senza deploy l'online resta indietro. Su lavori visivi vale anche il contrario — si deploia *per farglielo vedere*, non si aspetta una conferma che senza deploy nessuno può dare.

06 **Verifica guardando, non fidandosi**

Un HTTP 200 torna uguale anche su una build vecchia: si controlla l'età del deployment. Sul risultato di un agente si riverifica punto per punto, perché un agente riferisce cosa *intendeva* fare. C'è un browser headless in macchina: «non ho modo di vedere» non è una risposta ammessa.

Il cervello si aggiorna prima di chiudere

Scheda del progetto (stato, deciso, prossimo passo), lezione tecnica nell'archivio se supera tre criteri — riusabile fuori dal progetto, non ovvia, costata qualcosa da scoprire — e tre righe di riepilogo. Se la lezione non passa tutti e tre i criteri non entra: quel file è corto apposta.

I PUNTI DI APPOGGIO

Su cosa sta in piedi il sistema

Cinque appoggi, tre dei quali non sono codice. La memoria esterna è quella che rende sostenibile lavorare a turni: si riparte da dove si era rimasti anche dopo dieci giorni.

Il vault

`~/cervello/`

Identikit, regole d'ingaggio (254 righe, una ventina di lezioni tecniche datate e attribuite al progetto dove sono nate), schede di progetto, task pendenti, cinque skill richiamabili. È la fonte di verità su *come* si lavora, letta a ogni sessione.

Le skill

`06-skills/`

Procedure ripetibili invocate per nome: briefing mattutino da cron, chiusura sessione, pipeline newsletter, pipeline social. Trasformano un'abitudine in un passaggio che non si dimentica.

Il logbook nei prodotti

`The Crew` → `VVF`

Il canale di segnalazione è dentro l'applicativo, non fuori. Nato per la palestra, portato di peso sul gestionale dei vigili: quando una cosa funziona in un progetto, migra.

La produzione come banco di prova

`Fly.io` · `Vercel` · `Supabase`

Tutto è online e usato da persone vere: un comando dei vigili del fuoco, un'ASD con 82 soci, due prodotti pubblici. Nessuna demo, nessun cassetto.

L'AI come unico collaboratore

`88 file di memoria persistente`

Non un assistente a domanda-risposta: un esecutore diretto, con istruzioni permanenti scritte da lui — diretto, mai adulatorio, niente testo creativo al posto suo, e se una cosa è sbagliata lo si dice.

Che cosa gira, e su cosa

Sei sistemi, due famiglie tecnologiche distinte. Sotto ogni scheda ci sono le cifre verificate sul repository al 05/09/2026: non stime, conteggi.

vvf-gestionale Fogli di servizio, agenda assenze, amministrazione turni

IN PRODUZIONE

LINGUAGGIO

PHP 8.2 su Apache, in container. Nessun framework: pagine PHP e un motore proprio, **FoglioRenderer**, che dallo stesso modello genera anteprima HTML e file ODT.

DATABASE

TiDB Cloud (compatibile MySQL, gateway europeo, porta 4000), accesso via PDO. 37 tabelle: **vigili**, **assegnazioni**, **assenze**, **bot_requests**, **salto_servizio**, **fogli_servizio**...

HOSTING

Fly.io, regione Francoforte, 256 MB, macchina che si spegne da sola quando non serve. Sessioni PHP scritte sul database — altrimenti il riavvio della macchina fa uscire gli utenti a caso.

FORMATI

ODT (OpenDocument) riempito partendo dal modulo ufficiale vuoto, non ricostruito: le correzioni di geometria si fanno nel file **.odt**, mai con un ramo di codice dedicato.

DIMENSIONE

241 commit · ~34.000 righe PHP · dal 30/05/2026

vvf-ferie-bot

Canale Telegram: richieste, approvazioni, posta, backup

IN PRODUZIONE

LINGUAGGIO

Python 3 con `python-telegram-bot` 20.8 (job queue inclusa).

DATABASE

Lo stesso TiDB del gestionale, via `PyMySQL`. Il bot scrive solo richieste e stati; il foglio lo compone il gestionale.

SERVIZI

SMTP e **IMAP APPEND** su `vigilfuoco.it` — la copia in «Inviati» va scritta a mano, l'SMTP da solo non archivia niente. Google Drive e Sheets via `gsread` e OAuth. Cifratura credenziali con Fernet.

HOSTING

Fly.io Francoforte, 256 MB, volume persistente `/data`, macchina sempre accesa. Backup del database alle 04:00 → volume + cartella Drive + notifica Telegram.

DIMENSIONE

71 commit · dal 05/05/2026

the-crew

Gestionale ASD completo + sito pubblico — `thecrewgym.com`

IN PRODUZIONE

LINGUAGGIO

Next.js 16 (App Router) con React 19, TypeScript, Tailwind 4. Logica quasi tutta in Server Action, non in API.

DATABASE

Supabase / PostgreSQL, ed è qui che sta il peso vero del progetto: 66 tabelle, **161 policy di sicurezza a livello di riga**, 76 funzioni e 71 trigger. Estensioni `pgcrypto` e `citext`. Le regole di dominio — numerazione ricevute, conteggio ingressi, quote — vivono nel database, non nell'interfaccia.

PROCESSI AUTOMATICI

`pg_cron` dentro Postgres chiama sei rotte notturne: backup, generazione notifiche, chiusura mesi di compenso, bozza verbale, iscrizioni scadute, sentinella di controllo.

SERVIZI

Stripe con webhook per gli incassi · **Nodemailer** su SMTP Gmail con `imapflow` per la copia in Inviati · webhook **Telegram** per il bot dei collaboratori · `unpdf` per leggere gli estratti conto.

HOSTING

Vercel, regione `fra1`. PWA installabile con notifiche push.

DIMENSIONE

322 commit · ~52.000 righe TS/TSX · 134 migrazioni SQL numerate · dal 01/08/2026

br-turni · «Last Pact»

Battle royale a turni asincrono — last-pact.vercel.app

IN PRODUZIONE

ARCHITETTURA

Un **motore TypeScript puro** separato dall'interfaccia e copiato dentro il sito a ogni build: griglia esagonale, combattimento a intenti, bot, matchmaking, progressione. È l'unico progetto con una vera separazione fra regole e schermo.

VERIFICHE

254 test automatici con `vitest`, su 21 file. Più uno script `npm run shot` che produce screenshot veri del gioco — è così che sono venuti fuori un terzo di schermo vuoto e un accavallamento mai risolto.

SITO

Next.js 16, React 19, Supabase per gli account, `web-push` per il turno dell'avversario. Grafica generata con Gemini e rilavorata.

DIMENSIONE

206 commit · dal 19/07/2026

the-raven

Racconti AI a struttura ad albero — the-raven.it

FERMO DAL 19/06

LINGUAGGIO

Next.js 14 con React 18, Tailwind 3, `zod`, `framer-motion`, e `@xyflow/react` per disegnare e navigare l'albero della storia.

MODELLI AI

Non un fornitore solo ma un parco intercambiabile: **Llama 4 Scout**, **Llama 3.3 70B**, **gpt-oss-120b**, raggiunti via Groq, OpenRouter, Mistral e Gemini. La scelta del modello è una decisione di prodotto — gratuito per i minorenni, più capace per gli adulti.

LAVORI LUNGI

Trigger.dev 4.4.6: generare una storia dura minuti, quindi non può stare dentro una richiesta web.

SERVIZI

Supabase per dati e accessi · **Cloudflare R2** per i file · **Resend** per la posta · Google OAuth.

DIMENSIONE

305 commit · dal 12/03 al 19/06/2026

COME Python e `python-telegram-bot` su Fly.io. Un cron alle 7:30 costruisce il piano della giornata dai task pendenti e lo manda; se la macchina era spenta, lo recupera al riavvio.

Strumenti trasversali

POSTAZIONE	Chromebook con ambiente Linux — nessuna macchina di sviluppo tradizionale. Non esiste un ambiente locale: si prova direttamente sul deploy.
ESECUTORE	Claude Code da terminale, con collegamenti diretti (MCP) a Supabase, Vercel, Canva, Gmail, Google Calendar e Drive, e al browser Chrome.
VERIFICA VISIVA	Chrome headless con Playwright installato in locale: gli screenshot di ciò che è online si fanno, non si chiedono.
CODICE	GitHub via chiave SSH (l'accesso a token è stato revocato ad agosto dopo un'esposizione). Deploy: <code>fly deploy</code> per i due VVF, Vercel a ogni push per gli altri.
GRAFICA	Canva Pro, preso apposta perché venisse pilotato via API, non usato a mano.

Vincoli reali della piattaforma

Cose che il manuale non dice e che qui sono già costate tempo. Chi entra sul progetto le deve sapere prima, non scoprirle.

TiDB non è MySQL fino in fondo. Niente `AUTO_INCREMENT` come lo si intende di solito, e nel dump di backup `--single-transaction` non si può usare.

Il database dei vigili si interroga solo da dentro Fly, e `fly ssh console` è rotto: si passa da `machine exec` con il comando codificato in base64.

Una macchina Fly che si spegne da sola perde `/tmp` fra un comando SSH e il successivo. Copia e uso vanno fatti in una sola invocazione, o l'errore «file non trovato» sembra un bug di codice.

Le tabelle di log di Supabase durano poco: `net._http_response` conserva circa sei ore. Una diagnostica appoggiata lì sopra è cieca e non lo dice.

In serverless una notifica lanciata e dimenticata non parte. Si usa `after()`, ma i dati vanno letti prima — dopo la risposta il collegamento al database può non valere più.

La stessa regola scritta in PHP e ricopiata a mano nel JavaScript si disallinea da sola. La versione per il browser va generata dalle costanti del server: una sola fonte di verità.

Quello che nello stack non c'è

Nessuna integrazione continua, nessun ambiente di prova separato, nessuna infrastruttura descritta come codice, nessun controllo esterno che avvisi quando un servizio cade. I segreti vivono in file `.env` locali e nei pannelli dei fornitori, senza un inventario. Sono assenze coerenti fra loro: descrivono un sistema costruito per andare in produzione in fretta, non per essere mantenuto in due.

ANALISI

Dove il metodo regge

Consegna, e consegna in produzione

La misura non è il codice scritto ma il codice che qualcuno usa il giorno dopo. Cinque settimane per un gestionale d'associazione completo di quote, ricevute numerate, libro soci, PWA e notifiche.

PROVA

322 commit e 134 migrazioni su The Crew fra il 01/08 e il 05/09/2026. Sito pubblico live, soci reali dentro.

Gli errori diventano regole scritte, non buoni propositi

Ogni incidente produce una riga permanente che cambia il comportamento futuro — con la data, il progetto e il caso che l'ha generata. È il meccanismo che permette a un non-programmatore di non ripetere due volte lo stesso errore di sistema.

PROVA

«Un match *contains* su un placeholder può collidere con dati reali» — nata perché il cognome MORELLO contiene «ore» e spariva dai fogli di servizio.

Il dominio è vero e conosciuto dall'interno

I turni A/B/C/D, il salto turno, le patenti di guida dei mezzi, il foglio di servizio, i vincoli di un'ASD a norma. Nessun analista esterno arriverebbe a questo livello di dettaglio: qui il committente e l'utente sono la stessa persona.

PROVA

Un ruolo «Autista 2/3/4» dove una patente più grande copre il ruolo più piccolo ma la 1 no, e la stessa persona non può coprire due ruoli autista insieme.

Correggere la causa non basta: si impedisce l'effetto

Dopo un guasto che aveva spedito migliaia di email, il difetto è stato corretto e è stato messo un tetto a monte, con contatore su database e comportamento fail-closed: se non si può contare, non si spedisce.

PROVA

Migrazione 0110 (la causa) + 0111 (il tetto, max 3 email al giorno allo stesso indirizzo).

Sa contraddire l'AI, e ha ragione abbastanza spesso

Quando una diagnosi cozza con la sua esperienza diretta del sistema, la sua esperienza vince finché non c'è una prova contraria. Questa è una difesa reale contro il difetto principale di lavorare con un modello: la sicurezza con cui espone una spiegazione sbagliata.

PROVA

Regola permanente «causa giusta ≠ conclusione giusta»: prima di dire «X non funziona più», cercare la prova che X funziona ancora.

ANALISI

Dove il metodo si rompe

Nessuna di queste è un'ipotesi: sono guasti già avvenuti in produzione, tutti negli

ultimi trenta giorni, tutti su applicativi con utenti veri dentro.

Zero test automatici sui due applicativi che contano

Il gestionale dei vigili e quello dell'ASD non hanno un solo file di test, né una CI. Il gioco mobile — l'unico progetto senza utenti — ne ha 21. Il collaudo è manuale, dipende da chi lo esegue e non lascia traccia riutilizzabile.

CONSEGUENZA MISURATA

Le regole a calendario (cicli di turno, salto) sono state riscritte a mano in due punti e si sono disallineate da sole, producendo lo stesso bug due volte a distanza di un mese.

I difetti li trova l'utente, o il caso

Manca uno strato che si accorga da solo che qualcosa non ha funzionato. Le regressioni gravi sono emerse per segnalazione o per controllo incrociato fatto per altro motivo, giorni dopo.

CONSEGUENZA MISURATA

41 soci su 82 chiusi fuori dall'accesso per due giorni. 24 persone che avevano pagato risultavano scadute, con a schermo la frase giusta che prometteva il contrario.

Il collaudo si fa dentro la produzione

Dati finti scritti nel database vero e poi rimossi. Funziona finché chi pulisce si ricorda tutto: una riga «TEST» può avere agganciate ricevute numerate vere, e la numerazione fiscale non torna indietro.

CONSEGUENZA MISURATA

13 ricevute numerate reali da stornare, nate da un collaudo mai ripulito. Per alcuni minuti, dopo una copia di file isolata, tutte le ferie approvate sono apparse «in attesa» in produzione.

La delega agli agenti non ha una rete di sicurezza

Un agente con accesso diretto al database non è vincolato dalle protezioni scritte nell'interfaccia: la regola «questa azione resta sempre un click umano» vive nel codice applicativo, non nel DB. E un agente interrotto a metà lascia il lavoro *dichiarato* chiuso.

CONSEGUENZA MISURATA

Un verbale di ammissione confermato per davvero durante un collaudo — 14 secondi fra creazione e conferma — con 10 persone ammesse al libro soci, 9 delle quali senza aver pagato la quota.

Un solo punto di rottura, ed è una persona

Nessun altro essere umano può mettere le mani su questi applicativi. Non esiste documentazione di ingresso, ambiente di sviluppo riproducibile, né inventario delle credenziali. Il vault è ricchissimo di istruzioni *per l'AI* e quasi muto verso un collega.

CONSEGUENZA STRUTTURALE

Con l'ambizione dichiarata di rivendere il gestionale, ogni cliente oggi richiederebbe un progetto database separato: l'ostacolo è architetturale, non commerciale.

Si apre più di quanto si chiuda

Sei repository attivi in parallelo. Le priorità dichiarate a luglio — finire il gestionale VVF, mettere l'ASD a norma — a settembre sono ancora aperte, mentre due progetti nuovi sono nati nel frattempo e uno è fermo da giugno.

CONSEGUENZA MISURATA

The Raven: ultimo commit 19/06/2026, con un branch da unire e tre chiavi da ruotare in attesa da allora.

CONSIGLI

Sei mosse, in ordine di resa

Ordinate per rapporto fra ciò che risolvono e ciò che costano — il criterio dichiarato da lui stesso: massima resa per la minima spesa.

Test solo dove la logica è pura e sbagliarla si paga caro

COSTO BASSO

Non serve una suite: servono i cicli di turno, il salto, le scadenze degli abbonamenti e il calcolo dei compensi. Sono funzioni deterministiche, e sono esattamente i punti che hanno già prodotto guasti in produzione. La capacità c'è già — il gioco ne ha 21 file.

Ogni promessa fatta a schermo diventa un controllo sul dato

COSTO BASSO

Una frase che annuncia un effetto non è l'effetto. La domanda da porre a ogni funzionalità è « quanti casi hanno prodotto la frase e non l'effetto? », e la risposta va scritta in una query salvata, non cercata a mano quando qualcuno protesta.

Un registro proprio per ogni processo automatico

COSTO MEDIO

Il principio è già stato scoperto ed enunciato — *un sistema che si sorveglia da solo deve possedere le proprie tracce* — ma applicato finora a una cosa sola. Vale per il backup, per i job notturni, per gli invii: ogni corsa scrive il suo esito in una tabella propria, e una sentinella la legge.

Uno staging vero, invece dei dati finti in produzione

COSTO MEDIO

Gli strumenti in uso lo offrono già (i branch del database). Elimina in un colpo la classe di incidenti più costosa: quella in cui il collaudo stesso è il guasto.

Ai mandati per agenti va scritto il divieto, non solo l'obiettivo

COSTO MEDIO

Le azioni irreversibili vanno nominate esplicitamente come vietate nel brief, e il database va controllato dopo — mai il solo rapporto dell'agente. Meglio ancora: le protezioni «solo a mano» devono vivere nel database, dove un agente non le può scavalcare.

Chiudere prima di aprire

COSTO ALTO

La regola più difficile è quella che vale di più. Un progetto per volta fino allo stato «finito e a norma», e il divieto di iniziarne un altro prima. Il metodo di lavoro è già maturo; a mancare è il vincolo sul portafoglio di progetti.

Il patto, in chiaro

Da leggere prima della prima sessione, che si sia un'altra AI o una persona.

Cosa porta lui

- **Il dominio** — conosce dall'interno il mondo per cui costruisce, e la specifica arriva dettagliata al livello del caso limite.
- **La decisione** — sceglie in fretta e non torna indietro per abitudine; se contraddice una diagnosi, ha una ragione operativa.
- **La memoria del sistema** — vault, schede, lezioni. Chi entra non deve ricostruire niente: deve leggere.
- **Gli utenti** — c'è già chi usa quello che si costruisce, quindi il riscontro arriva in ore, non in trimestri.

Cosa si aspetta

- **Diretto e alla pari.** Niente preamboli, niente adulazione, niente spiegazione delle basi. Se una cosa è sbagliata, si dice subito.
- **Risultati, non opzioni.** Una raccomandazione, non un ventaglio. Il tempo è a turni: le sessioni sono corte e distanziate.
- **Consegna, non promessa.** Il lavoro non è finito al commit: è finito quando è online e verificato guardandolo.
- **Un limite invalicabile:** il testo creativo è suo. Non si scrive al posto suo, non si riformulano le sue frasi. Vale per la narrativa e per la sua voce personale, non per il copy commerciale.

Dove serve davvero aiuto

- **Ingegneria della qualità** — test, CI, staging, sentinelle: tutto il livello che oggi manca per intero.
- **Architettura multi-cliente** — il passaggio da un gestionale che funziona a un prodotto vendibile a più organizzazioni.
- **Documentazione per umani** — la condizione perché esista una seconda persona sul progetto, oggi assente.

Documento di riferimento interno — profilo di metodo, non curriculum narrativo.

Fonti: cronologia git dei sei repository (marzo-settembre 2026), regole d'ingaggio e archivio lezioni in ~/cervello/, schede di progetto, 88 file di memoria persistente. Tutte le cifre sono verificate alla data del 05.09.2026. Gli incidenti citati sono documentati con data e progetto d'origine.